

Step 12 — Wire Up WarehouseFilter

logistiq · frontend, first real feature after deployment

What this fixes: the dropdown on your dashboard is a fully working piece of UI that doesn't do anything — it holds its own local "selected" state and never tells the page anything changed. It also lists three fake warehouses ("Warehouse A" , "Warehouse B" , "Warehouse C") instead of your real ones.

Scope: which cards actually change

I went through every card on the dashboard before touching anything, because not everything on that page is warehouse-scoped data. Only two are:

- **Products to reorder** — stock quantity is tracked per-warehouse in your schema, so "low stock" can honestly mean "low stock in this warehouse" once filtered.
- **Recent inventory activity** — inventory events already carry a `warehouseId` ; the backend handler for this already accepts a filter, it just wasn't being used.

Top products, top customers, open purchase orders, unpaid sales orders, and the revenue chart stay exactly as they are — those are organization-wide by nature (a customer or a sale doesn't belong to one warehouse in this schema), so filtering them by warehouse would be fabricating a relationship that isn't real, not "increasing effort." Same for the "X SKUs tracked" count in the header — a product/SKU isn't confined to one warehouse, so that number is intentionally left showing the org-wide total either way.

1 Let `lowStock` accept a warehouse filter

This backend action currently sums a product's stock across every warehouse, with no way to scope it to one. Add that.

`app/modules/inventory/inventory.ts` — find the "lowStock" handler and replace it with:

```
register("lowStock", async (data, ctx) => {
  const { warehouseId } = data;

  const products = await prisma.product.findMany({
    where: { organizationId: ctx.organizationId },
    include: { inventoryItems: true },
  });

  const lowStock = products
```

```

    .map((p) => ({
      ...p,
      totalQuantity: p.inventoryItems
        .filter((i) => !warehouseId || i.warehouseId === warehouseId)
        .reduce((sum, i) => sum + i.quantity, 0),
    }))
    .filter((p) => p.totalQuantity <= p.reorderPoint);

    return { status: 200, body: { products: lowStock } };
  })
}

```

Two changes from what's there now: the handler reads `data` instead of ignoring it (`_data` → `data`), and the quantity sum filters to `warehouseId` when one's given, otherwise behaves exactly as before. `listInventoryEvents` needs no change — it already accepts `warehouseId`, it just wasn't being passed one from the dashboard.

2 Let ACCOUNTING actually see the filter

Small thing I caught while checking permissions, unrelated to the two changes above but adjacent enough to fix here: every other card on this dashboard is visible to the `ACCOUNTING` role, but `listWarehouse` — the action that will populate this dropdown — isn't in its allowed list. That wouldn't break anything (a blocked call just falls back to an empty list on this page), but `ACCOUNTING` users would see a dropdown with no warehouses in it, silently. Bring it in line with the rest of the dashboard's permissions.

`app/lib/permission.ts` — change this line:

```
listWarehouse: [...ALWAYS_ALLOWED, "WAREHOUSE_STAFF", "PURCHASING", "VIEWER"],
```

to:

```
listWarehouse: [...ALWAYS_ALLOWED, "WAREHOUSE_STAFF", "PURCHASING",
  "ACCOUNTING", "VIEWER"],
```

3 Read the filter from the URL, fetch real warehouses

The dashboard is a Server Component — it fetches all its data on the server, before the page ever reaches the browser. So "which warehouse is selected" needs to live somewhere the server can read it on each request: the URL's query string (`?warehouse=<id>`), not React state. This also means the filter survives a refresh or a shared link, which local state never would.

`app/dashboard/page.tsx` — change the function signature from:

```
export default async function DashboardPage() {
  const user = await getCurrentUser();
```

to:

```
export default async function DashboardPage({
  searchParams,
}: {
  searchParams: Promise<{ warehouse?: string }>;
}) {
  const { warehouse: warehouseId } = await searchParams;
  const user = await getCurrentUser();
```

(`searchParams` is a Promise in this Next.js version — same pattern as everything else already async on this page.)

Then, in the `Promise.all` block, change this:

```
const [lowStockRes, purchaseOrdersRes, topProductsRes, topCustomersRes,
revenueRes, unpaidRes, activityRes, productsRes] =
  await Promise.all([
    orchestrate({ action: "lowStock" }, ctx),
    orchestrate({ action: "listPurchaseOrders" }, ctx),
    orchestrate({ action: "topProducts", limit: 3 }, ctx),
    orchestrate({ action: "topCustomers", limit: 3 }, ctx),
    orchestrate({ action: "revenueByDay", days: 30 }, ctx),
    orchestrate({ action: "listSalesOrders", unpaidOnly: true }, ctx),
    orchestrate({ action: "listInventoryEvents", limit: 5 }, ctx),
    orchestrate({ action: "listProducts" }, ctx),
  ]);
```

to:

```
const [lowStockRes, purchaseOrdersRes, topProductsRes, topCustomersRes,
revenueRes, unpaidRes, activityRes, productsRes, warehousesRes] =
  await Promise.all([
    orchestrate({ action: "lowStock", warehouseId }, ctx),
    orchestrate({ action: "listPurchaseOrders" }, ctx),
    orchestrate({ action: "topProducts", limit: 3 }, ctx),
    orchestrate({ action: "topCustomers", limit: 3 }, ctx),
    orchestrate({ action: "revenueByDay", days: 30 }, ctx),
    orchestrate({ action: "listSalesOrders", unpaidOnly: true }, ctx),
    orchestrate({ action: "listInventoryEvents", limit: 5, warehouseId },
ctx),
    orchestrate({ action: "listProducts" }, ctx),
```

```
    orchestrate({ action: "listWarehouse" }, ctx),
  });
```

Then, right after the existing `totalSkuCount` line, add:

```
const warehouses = bodyArray<{ id: string; name: string }>(warehousesRes,
"warehouses");
```

Finally, find where the component is rendered and change:

```
<WarehouseFilter />
```

to:

```
<WarehouseFilter warehouses={warehouses} selectedId={warehouseId} />
```

4 Rewrite WarehouseFilter to actually filter

The dropdown UI itself doesn't need to change — only what happens on click. Instead of setting local state, clicking an option now updates the URL's `warehouse` query param and navigates, which re-runs the Server Component above with the new value.

`app/components/dashboard/WarehouseFilter.tsx` — replace the whole file with:

```
"use client";

import { useEffect, useRef, useState } from "react";
import { usePathname, useRouter, useSearchParams } from "next/navigation";
import { MapPin, ChevronDown, Check } from "lucide-react";

type Warehouse = { id: string; name: string };

export default function WarehouseFilter({
  warehouses,
  selectedId,
}: {
  warehouses: Warehouse[];
  selectedId?: string;
}) {
  const [open, setOpen] = useState(false);
  const ref = useRef<HTMLDivElement>(null);
  const router = useRouter();
  const pathname = usePathname();
  const searchParams = useSearchParams();
```

```

useEffect(() => {
  function onClickOutside(e: MouseEvent) {
    if (ref.current && !ref.current.contains(e.target as Node))
setOpen(false);
  }
  document.addEventListener("mousedown", onClickOutside);
  return () => document.removeEventListener("mousedown", onClickOutside);
}, []);

const selected = warehouses.find((w) => w.id === selectedId);
const label = selected ? selected.name : "All Warehouses";

function select(id: string | null) {
  const params = new URLSearchParams(searchParams.toString());
  if (id) {
    params.set("warehouse", id);
  } else {
    params.delete("warehouse");
  }
  const query = params.toString();
  router.push(query ? `${pathname}?${query}` : pathname, { scroll: false });
  setOpen(false);
}

return (
  <div className="relative" ref={ref}>
    <button
      type="button"
      onClick={() => setOpen((v) => !v)}
      className="flex items-center gap-2 rounded-full border border-slate-200
bg-white pl-3.5 pr-3 py-2 text-sm font-semibold text-foreground shadow-sm
hover:border-slate-300 transition-colors"
    >
      <MapPin size={14} className="text-accent" />
      {label}
      <ChevronDown size={14} className={`text-slate-400 transition-transform $
{open ? "rotate-180" : ""}`} />
    </button>

    {open && (
      <div className="absolute right-0 sm:left-0 top-full z-20 mt-2 w-48
rounded-xl border border-slate-200/70 bg-white shadow-xl shadow-slate-900/[0.08]
p-1.5">
        <button
          type="button"
          onClick={() => select(null)}
          className="w-full flex items-center justify-between gap-2 rounded-lg
px-3 py-2 text-sm font-medium text-foreground hover:bg-slate-50 transition-

```

```

colors"
  >
    All Warehouses
    {!selected && <Check size={14} className="text-accent" />}
  </button>
  {warehouses.map((w) => (
    <button
      key={w.id}
      type="button"
      onClick={() => select(w.id)}
      className="w-full flex items-center justify-between gap-2 rounded-
lg px-3 py-2 text-sm font-medium text-foreground hover:bg-slate-50 transition-
colors"
    >
      {w.name}
      {selected?.id === w.id && <Check size={14} className="text-
accent" />}
    </button>
  ))}
</div>
)}
</div>
);
}

```

Two behavior notes worth knowing, not just copying blindly: `{ scroll: false }` on `router.push` stops the page from jumping back to the top every time you pick a warehouse — without it, App Router's default is to scroll to top on navigation, which feels broken on a filter click even though nothing's actually wrong. And selecting "All Warehouses" removes the query param entirely rather than setting it to some sentinel value — so the URL stays clean (`/dashboard` , not `/dashboard?warehouse=all`) when nothing's filtered.

5 Verify

```

npx tsc --noEmit
npm run build
npx vitest run

```

I verified this first: applied all four changes above to a throwaway copy of your project, not your real files. `tsc --noEmit` passed clean — zero errors, exit code 0 (there's nothing like the stray `types/` folder issue this time; that's already fixed in your real repo). I also checked `permission.test.ts` by hand — it only asserts against `createProduct` and a made-up action

name, so the `listWarehouse` permission change doesn't touch anything it tests. I still couldn't run your actual `next build` or `vitest` here (same architecture mismatch as before), so once you apply this, run the three commands above yourself and flag anything that doesn't match.

After this

Once verified locally, commit, push, and Vercel will redeploy automatically — no dashboard steps needed this time, it's already connected. Try the filter on the live site once it's up: pick a warehouse, confirm "Products to reorder" and "Recent inventory activity" actually change, and that the other cards correctly stay the same.

With this done, both structural gaps from the original review — test coverage and the disconnected filter — plus deployment are closed out.